DBAAS (MARIADB)
SELF-SERVICE ADMIN PROCEDURE
USER GUIDE
OF
GOVERNMENT CLOUD
INFRASTRUCTURE SERVICES
(GCIS)
FOR
THE OFFICE OF THE GOVERNMENT CHIEF INFORMATION OFFICER

Version: 1.5

DEC 2021

Amendment History

Change Number	Revision Description	Pages Affected on Respective Version	Revision / Version Number	Date	Approval Reference
	Initial		1.0	15-Oct-2020	
	Update YUM address of P1 and P2 Add JDBC connection parameter	6 14	1.1	19-Oct-2020	
	Update database client tool to connect DB and add screenshot of connection setting of one client tool as example of GUI client	5, 6	1.2	09-Dec-2020	
	● Update wording related to auto-incremental value ● Update database user role privileges ● add section 7.4 about mariadb-dump parameter for Active-Active DBaaS ● add section 9 for non-default database parameters ● Correct the timeout term “connection timeout” to “connection idle timeout” ● Improve the message of patching maintenance for Active-Active DBaaS	4, 8, 9, 14, 16	1.3	18-Mar-2021	
	● Allow drop schema if schema has DB user associate with it ● Change role to initial grant in create DB user procedure ● Remove expiry parameter in create DB user procedure ● Add section to create never expire read-write DB user ● Remove change DB user procedure ● Add procedure to check DB locking, DB blocking, flush privileges and flush host cache ● Update object owner privileges ● Add Sequence db object related limitation in Active-Active DBaaS ● Update lock limitation in Active-Active DBaaS	4, 7, 8, 9, 10, 11, 12, 13, 14	1.4	04-Aug-2021	
	● Restructure the whole document ● Swap section of abbreviation and points to note ● Change section 6 “Basic DBaaS Administration” to Appendix ● Revise Points to note section about new Primary Key enforcement, Sequence database object and DDL SQL ● Specify SSP portal should be used to create database schemas and users instead of self-service admin procedures ● Update privilege of initial role ObjectOwner, APP_ReadWrite and ReadWrite ● Revise recommended mariadb-dump parameters ● Add section to emphasis using small transaction ● Specify B/D can raise “Other Request” in case wants to change DB parameter values	4 to 17	1.5	04-Oct-2021	

TABLE OF CONTENTS

1. PURPOSE

This document demonstrates the usage of GCIS DBaaS (MariaDB) self-service admin procedures, which serves as a guide to manage GCIS DBaaS (MariaDB) database schemas and users for application use.

2. ABBREVIATION AND ACRONYM

Abbreviation	Description
B/Ds	Bureaux and Departments
DBaaS	Database as a Service
GCIS	Government Cloud Infrastructure Services
SQL	Structured Query Language
SSP	Self-Service Portal

3. POINTS TO NOTE

1. For Java application, MariaDB Connector/J JDBC driver should be used in order to connect with the DBaaS. For details, please refer to (https://mariadb.com/kb/en/about-mariadb-connector-j/). Connection string is described in later part of this user guide.

2. Please read GCIS DBaaS house rules for other details.

3. The following database parameters have been set to non-default values, which may have impact to the application design.

Parameter Name	Value
autocommit	0
lower_case_table_names	1
transaction-isolation	read-committed

4. In Active-Active DBaaS, two-node database cluster architecture is used. All database connections are connected to one database node in normal situation and the other database is served as hot standby. In case of database failure, application can switch over to another healthy database in a short time automatically.

5. Below are the limitations of Active-Active DBaaS.

   a) Primary key is enforced in all tables in Active-Active DBaaS. If a new table is created without primary key, error “ERROR 1173 (42000) will be prompted.

   b) Optimistic replication is used for the data synchronization between database nodes and it can guarantee zero Recovery Point Objective (RPO). In the replication protocol, all database write operations are propagated to other database nodes at the commit time and conflict resolution is performed after that. In case there is data conflict, the transaction will be rollback. Because we have redirected all database connections to one node only in normal situation, rollback caused by conflict resolution during commit is minimized.

   c) As the write operations are propagated to other database nodes at the commit time, it is not recommended to create a large transaction in Active-Active DBaaS as it may affect the database performance seriously. If there is a large data purging operation, it is recommended to break down into segmented transaction with multiple commit points.

d) As we cannot guarantee all database connections are connected to only one database in all the time, “select for update” and explicit table lock may not work as expected because SHARE mode lock is not shared across all database nodes.

e) Active-Active DBaaS uses Total Order Isolation (TOI) to replicate DDL statements such as create/alter table across database nodes. It will block all write operations during the replication. Normally, the DDL replication takes a very short to complete if the DDL is a create/drop statement. However, if the table size is large and the DDL operation is to change the column as a primary key, it will block all write operations in the database until the primary key is created.

f) Please read Section 8.4 before using sequence object or auto-increment in Active-Active DBaaS because the sequence number may be generated by two database nodes.

g) Active-Active DBaaS cannot support XA transaction. For example, TransactionScope in .NET

h) Other minor limitations are listed in https://mariadb.com/kb/en/mariadb-galera-cluster-known-limitations/.

4. BASIC INFORMATION FOR DBAAS CONNECTION

4.1 ADMINISTRATIVE DATABASE USER ACCOUNT

After DBaaS Provisioning, an administrative database user account in the form of {gcis_id}_dba is created for accessing the DBaaS. This user account has sufficient privileges to create schema/database, create user account, change user account privileges, manage user database session by calling predefined administrative stored procedures.

DB Admin User Name	{gcis_id}_dba
DB Admin User Password	{provided after DBaaS provisioning}

4.2 ENDPOINTS OF THE DBAAS

Normally, all endpoints of the DBaaS are given in the form of URL with domain name.

In Active-Active DBaaS, 4 endpoints are provided for normal application transactions and 2 endpoints are provided for read-only operation such as reporting. If B/Ds use Java application with Connector/J driver, user should use load-balancing features provided by the driver in order to achieve the high availability. If B/Ds use command line tools to run SQL commands, users should use one of the endpoints to connect the database.

Active-Active DBaaS Endpoints Example
For application use, t1vdbs-dept1-testdb1-ha1.dbaas.gcisdctr.hksarg t1vdbs-dept1-testdb1-ha2.dbaas.gcisdctr.hksarg t2vdbs-dept1-testdb1-ha1.dbaas.gcisdctr.hksarg t2vdbs-dept1-testdb1-ha2.dbaas.gcisdctr.hksarg For read-only operation such as reporting, t1vdbs-dept1-testdb1-rpt.dbaas.gcisdctr.hksarg t2vdbs-dept1-testdb1-rpt.dbaas.gcisdctr.hksarg

In Active-Passive DBaaS, one endpoint is provided.

Active-Passive DBaaS Endpoints Example
t0vdbs-dept1-testdb1.dbaas.gcisdctr.hksarg

The database port number for the connection of both Active-Active and Active-Passive is 19307

DB Port	19307

5. CONNECTING TO DBAAS

5.1 FIREWALL RULES

There is no network permission from B/Ds servers to the DBaaS by default. Users should submit firewall requests to enable the network connection between tenant's VM and DBaaS.

5.2 CONNECTION TOOLS

For MariaDB, MariaDB client tools is always used by the application users to run the database commands. Users should install the client tools in their VM by themselves.

For Linux, users can install the tools via the YUM server provided by GCIS. YUM repo configuration is already configured in GCIS IaaS VM RedHat template.

In case yum repo file /etc/yum.repos.d/mariadb.repo does not exist (for example, not using GCIS VM template), please create the repo file /etc/yum.repos.d/mariadb.repo with the following contents

Testing Site:

```
[MariaDB]
name=MariaDB
baseurl=https://mariadb-yum.cs.gcisdctr.hksarg/mariadb-repo/mariadb-repo/
enabled=1
gpgcheck=1
sslverify=0
gpgkey=https://mariadb-yum.cs.gcisdctr.hksarg/mariadb-repo/mariadb-repo/RPM-GPG-KEY-
MariaDB
```

P1/P2 Site:

```
[MariaDB]
name=MariaDB
baseurl=https://mariadb-yum.cs.gcis.hksarg/mariadb-repo/mariadb-repo/
enabled=1
gpgcheck=1
sslverify=0
gpgkey=https://mariadb-yum.cs.gcis.hksarg/mariadb-repo/mariadb-repo/RPM-GPG-KEY-MariaDB
```

If the YUM repo configuration is properly set, run the following command to install MariaDB client tools as root.

```
# yum install MariaDB-client MariaDB-common MariaDB-compat
```

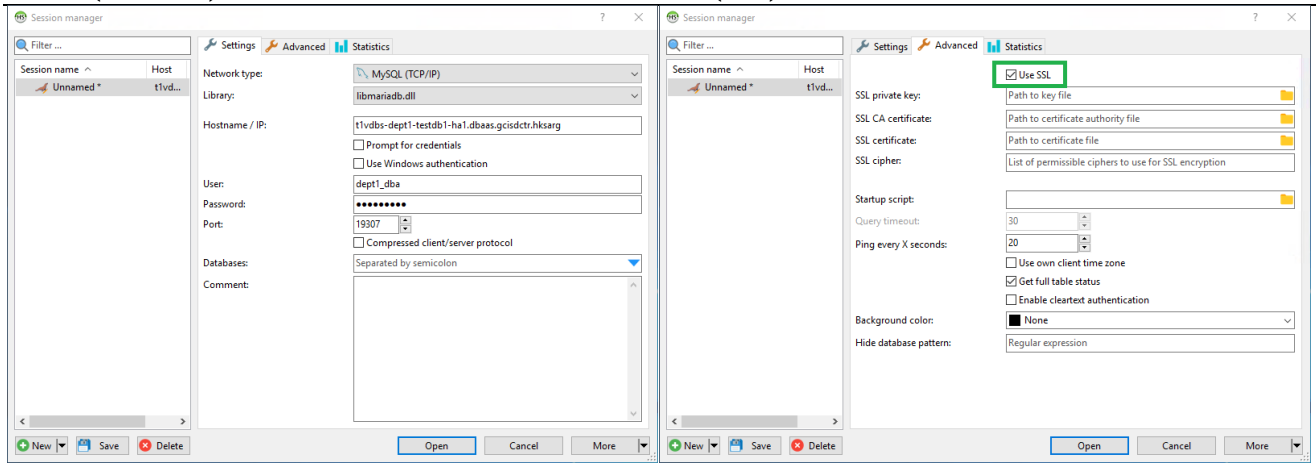
When YUM asks for importing GPG key, please reply "y"

```
warning: /var/cache/yum/x86_64/7Server/MariaDB/packages/MariaDB-client-10.5.5-
1.el7.centos.x86_64.rpm: Header V4 DSA/SHA1 Signature, key ID 1bb943db: NOKEY
Retrieving key from https://mariadb-yum.cs.gcisdctr.hksarg/mariadb-repo/mariadb-
repo/RPM-GPG-KEY-MariaDB
Importing GPG key 0x1BB943DB:
 Userid     : "MariaDB Package Signing Key <package-signing-key@mariadb.org>"
 Fingerprint: 1993 69e5 404b d5fc 7d2f e43b cbcb 082a 1bb9 43db
 From       : https://mariadb-yum.cs.gcisdctr.hksarg/mariadb-repo/mariadb-repo/RPM-GPG-
KEY-MariaDB
Is this ok [y/N]: y
```

For Windows clients or Linux without using YUM server, users can download packages (rpm, deb) from the below webpage and install the packages.

https://downloads.mariadb.org/mariadb/

Alternatively, in windows, users can connect DB using GUI client listed in https://mariadb.com/kb/en/graphical-and-enhanced-clients/. Please refer to documentation of GUI client you choose. Below is connection setting of HeidiSQL as an example.



5.3 EXAMPLE USING MARIADB CLIENT

The TLS 1.2 has been enabled for all database connections to the DBaaS. After the database provisioning, a B/D admin user account has been created. Users can connect to the database by the user via the MariaDB client tools with the following command line syntax:

mariadb -h [Address] -P [Port] -u [DB admin user] -p --ssl

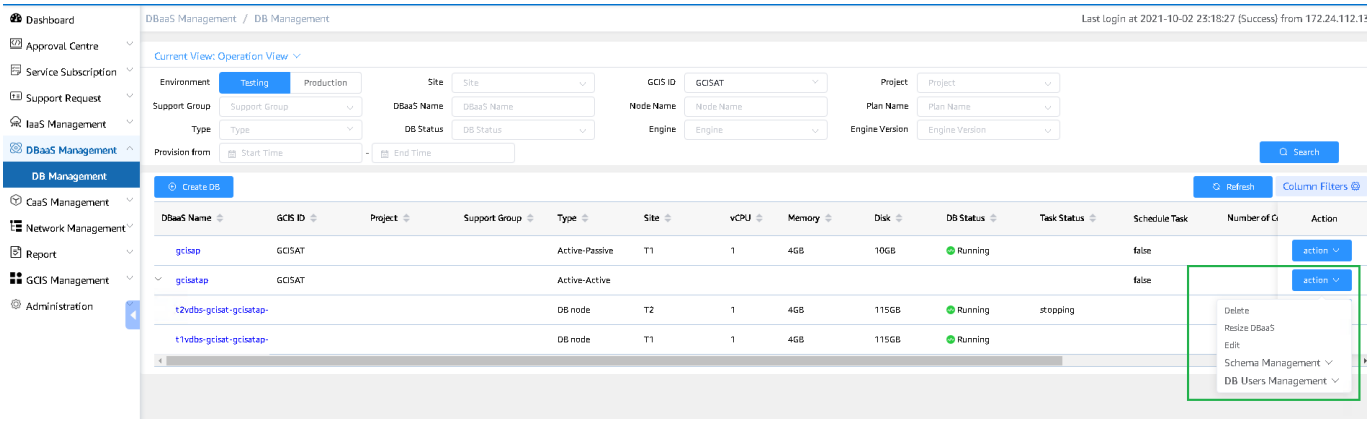
Note: for DBaaS Active-Active, any one from provided addresses can be used for schema setup.

Example:

```
$ mariadb -h t1vdbs-dept1-testdb1-ha1.dbaas.gcisdctr.hksarg -P 19307 -u dept1_dba -p --ssl
Enter password: (enter password here)
Welcome to the MariaDB monitor.  Commands end with ; or \g.
Your MariaDB connection id is 29198
Server version: 10.5.4-MariaDB-log MariaDB Server

Copyright (c) 2000, 2018, Oracle, MariaDB Corporation Ab and others.

Type 'help;' or '\h' for help. Type '\c' to clear the current input statement.

MariaDB [(none)]>
```

6. BASIC DBAAS ADMINISTRATION

GCIS portal provides Schema Management and DB Users Management functions for B/Ds. This is the preferable way for B/Ds to create and manage schema, user account and database.



DBaaS also provides a set of predefined store procedures for application to manage the DBaaS, such as change database user password, list database sessions, list database lock, kill database session, flush privilege, flush host cache, etcs. Please refer to Appendix A for details.

7. CONNECTING DBAAS FROM APPLICATION

7.1 JAVA

For Java application, please use MariaDB Connector/J version 2.6.1 or above
https://mariadb.com/kb/en/about-mariadb-connector-j/

Please read below link about JDBC connection string details
https://mariadb.com/kb/en/failover-and-high-availability-with-mariadb-connector-j/

Please set JDBC parameter "useSSL" and "trustServerCertificate" to true.

Below is example JDBC connection string for DBaaS Active-Passive.

```
jdbc:mariadb://t0vdbs-dept1-
testdb1.dbaas.gcisdctr.hksarg:19307/testdb1?useSSL=true&trustServerCertificate=true
```

Below is an example JDBC connection string for DBaaS Active-Active connection to read-write end points with load balance mode.

```
jdbc:mariadb:loadbalance://t1vdbs-dept1-testdb1-ha1.dbaas.gcisdctr.hksarg:19307,t1vdbs-
dept1-testdb1-ha2.dbaas.gcisdctr.hksarg:19307,t2vdbs-dept1-testdb1-
ha1.dbaas.gcisdctr.hksarg:19307,t2vdbs-dept1-testdb1-
ha2.dbaas.gcisdctr.hksarg:19307/testdb1?useSSL=true&trustServerCertificate=true
```

Below is an example, for connection string to read-only end points using load balance mode.

```
jdbc:mariadb:loadbalance://t1vdbs-dept1-testdb1-rpt.dbaas.gcisdctr.hksarg:19307,t2vdbs-
dept1-testdb1-
rpt.dbaas.gcisdctr.hksarg:19307/report?useSSL=true&trustServerCertificate=true
```

7.2 WINDOWS .NET

For Windows .NET application, please use MySqlConnector
https://mariadb.com/kb/en/mysqlconnector-for-adonet/
https://www.nuget.org/packages/MySqlConnector/

7.3 OTHERS

For other DB client libraries, please check the compatibility with MariaDB the following 2 links
https://downloads.mariadb.org/mariadb/10.5.5/
https://mariadb.com/kb/en/client-libraries/

7.4 MARIADB-DUMP PARAMETER FOR DBAAS

The preferred way to load data into DBaaS is by breaking down a single INSERT SQL statement with a large number of rows into multiple INSERT SQL statements and commit each statement. Below is an example to export data from an DBaaS using default multiple-row feature of command mariadb-dump (or symlink mysqldump). Every single large table will be exported into multiple INSERT SQL statements. mariadb-dump parameter "net-buffer-length" is used to control the transaction size of each INSERT SQL statement in order to improve the performance.

Another mariadb-dump parameter "--single-transaction" is used to export large tables without blocking other applications.

Please refer to https://mariadb.com/kb/en/mysqldump/ for more details on export option.

Example of export in Active-Passive DBaaS:

```
mariadb-dump -h t1vdbs-dept1-testdb1-ha1.dbaas.gcisdctr.hksarg -P 19307 -u app_owner -p
--ssl --databases testdb1 --net-buffer-length=16777216 --single-transaction > dump.sql
```

Active-Active DBaaS does not support lock table. Thus, when the destination DB is Active-Active DBaaS, please specify mariadb-dump (or symlink mysqldump) parameter "--skip-add-locks". In case this parameter is not used in export, you need to update the dump file in order to remove "LOCK TABLE" command before importing.

Example of export in Active-Active DBaaS:

```
mariadb-dump -h t1vdbs-dept1-testdb1-ha1.dbaas.gcisdctr.hksarg -P 19307 -u app_owner -p
--ssl --databases testdb1 --skip-add-locks --net-buffer-length=16777216 --single-
transaction > dump.sql
```

When importing data using auto-commit, it will commit each SQL statement to improve the performance. The below example of import using auto-commit applies to both Active-Passive and Active-Active DBaaS.

Example of import:

```
mariadb -h t1vdbs-dept1-testdb1-ha1.dbaas.gcisdctr.hksarg -P 19307 -u app_owner -p --
ssl --database testdb1 --init-command="set autocommit=1;" < dump.sql
```

8. OTHER DBAAS OPERATIONS

8.1 DATABASE PATCHING

GCIS DBaaS will have regular patching exercise, user can use SSP to schedule the patch time.

DBaaS Active-Passive requires 15mins to 30mins down time to patch database.

DBaaS Active-Active uses rolling upgrade. Patch is applied one node by one node. Endpoints will redirect DB requests to the DB node still online. There is no service down time during patching provided that the database connection idle timeout is set less than 2 hours and the application can reconnect to database automatically in the node redirection.

8.2 DATABASE RESTORATION

GCIS DBaaS only supports manual database restoration. Users can make request to restore their database to a given point-in-time. Other Request in SSP should be used to submit the restore request. A new temporary DB will be created with restored data within agreed time. Separate endpoint will be provided for user to get data back. Afterward, the temporary DB will be removed.

8.3 DISASTER RECOVERY

GCIS DBaaS supports cross-site high availability in production. In case of disaster, DB service will be still available in the remaining site.

For DBaaS Active-Passive, DB will be failed-over to the remaining site. DB service interruption is 5 to 10 minutes.

For DBaaS Active-Active, each site has one DB node online. In the remaining site, there is always a DB node online. Endpoints will redirect DB request to the DB node still online. DB service interruption is a few seconds.

8.4 SEQUENCE OBJECT AND AUTO-INCREMENT COLUMN

There are some constraints and limitations when using numeric value generated from sequence object or auto-increment column as a primary key in Active-Active DBaaS:

1. The numeric values generated are not in sequential (i.e. 1, 2, 3...). The pattern of the numeric value generated depends on which DB node is connected. For example, if it is connecting to DB node 1, the numeric value generated will be in the pattern 1, 4, 7, 10... If it is connecting to DB node 2, the pattern will be 2, 5, 8, 11... Hence the numeric values generated will be unique for all DB nodes.

2. INCREMENT=0 parameter must be used in order to create sequence object in Active-Active DBaaS. This can ensure the sequence number generated is unique for all DB nodes.

   Example:
   ```
   CREATE SEQUENCE s INCREMENT=0;
   ```

3. "ALTER SEQUENCE" command has a bug in Active-Active DBaaS. Instead, please use "DROP SEQUENCE" and then "CREATE SEQUENCE" with a larger starting value using "START WITH" parameter to make sure new sequence number generated is unique. The starting value should be greater than the maximum of existing sequence number.

   Example:
   ```
   DROP SEQUENCE s;
   CREATE SEQUENCE s START WITH {no. greater than existing max. seq. no.} INCREMENT=0;
   ```

4. Using mysql-dump to export data of table using sequence object will generate SQL statement to re-create the sequence object with a new starting value (see SELECT SETVAL(...) in the below example).

   Example:

```
DROP SEQUENCE IF EXISTS 's';
CREATE SEQUENCE `s` start with 1 minvalue 1 maxvalue 9223372036854775806 increment
by 0 cache 1000 nocycle ENGINE=InnoDB;
SELECT SETVAL(`s`, 27001, 0);
```

   When the SQL statement is loaded into the DB node, it also resets the sequence object in the remote DB node starting from beginning. The new sequence value generated in the remote DB node will have a chance of colliding with existing sequence numbers. In order to avoid the problem, the "START WITH" parameter should be changed from 1 to a value greater than existing maximum sequence value and the "SELECT SETVAL(…)" statement should be removed.

   Example:

```
DROP SEQUENCE IF EXISTS 's';
CREATE SEQUENCE `s` start with {no. greater than existing max. seq. no.} minvalue 1
maxvalue 9223372036854775806 increment by 0 cache 1000 nocycle ENGINE=InnoDB;
```

9. CHANGEABLE DATABASE PARAMETERS

Please refer to MariaDB documentation about the description of each parameter
https://mariadb.com/kb/en/server-system-variables/
https://mariadb.com/kb/en/innodb-system-variables/

A self-admin function will be available in the GCIS SSP to allow tenants to change the following parameter values. In the meantime, users can submit "Other Change" in SSP to do so.

Parameter	Value in GCIS DBaaS
innodb_buffer_pool_size	Subscription plan memory size x 25%
max_connections	400 – subscription plan DBS, DBM 800 – subscription plan DBL, DBX, etc.
autocommit	0
max_allowed_packet	64M
sort_buffer_size	2M
read_buffer_size	1M
innodb_thread_concurrency	8
explicit_defaults_for_timestamp	ON
lower_case_table_names	1
transaction-isolation	read-committed
wait_timeout	3,600
log_bin_trust_function_creators	1
query_cache_type	ON
query_cache_size	64M
innodb_autoinc_lock_mode	2
innodb_lock_wait_timeout	50
innodb_status_output_locks	ON
max_heap_table_size	67108864
tmp_table_size	67108864

APPENDIX A. STORED PROCEDURES FOR DBAAS ADMINISTRATION

Application can use the following self service admin stored procedures to manage DBaaS.

APPENDIX A.1 CREATE DATABASE/SCHEMA

Syntax:

```
call GCIS_SCHEMA.CREATE_DB_SCHEMA('[Schema Name]');
```

Restrictions:

1. Schema name can be combination of numeric and alphabetic characters. Symbol characters are not allowed
2. Schema name cannot be the same name to any existing schema
3. Schema name cannot begin with “gcis”, “mysql” or “mariadb”
4. Length of database schema name is 64. It is equivalent to object type “database” in MariaDB terminology. Refer: https://mariadb.com/kb/en/identifier-names/

Example:

Create database/schema with name “testdb1”.

```
MariaDB [(none)]>call GCIS_SCHEMA.CREATE_DB_SCHEMA('testdb1');
+----------------------------+
| Message                    |
+----------------------------+
| Success, schema is created. |
+----------------------------+
1 row in set (0.040 sec)
```

APPENDIX A.2 LIST DATABASE/SCHEMA

Syntax:

```
call GCIS_SCHEMA.LIST_DB_SCHEMA();
```

Example:

List all schema created by the user.

```
MariaDB [(none)]>call GCIS_SCHEMA.LIST_DB_SCHEMA();
+-------------+
| schema_name |
+-------------+
| testdb1     |
+-------------+
1 row in set (0.002 sec)
```

APPENDIX A.3 DROP DATABASE/SCHEMA

Syntax:

```
call GCIS_SCHEMA.DROP_DB_SCHEMA('[Schema Name]');
```

Restrictions:

1. Must be existing DB schema
2. Cannot drop DB system schema

Example:

```
MariaDB [(none)]>call GCIS_SCHEMA.DROP_DB_SCHEMA('testdb1');
+----------------------------+
| Message                    |
+----------------------------+
| Success, schema is dropped. |
+----------------------------+
1 row in set (0.007 sec)
```

APPENDIX A.4 CREATE DATABASE USER WITH "OBJECT OWNER" PRIVILEGE

16 schema privileges and one procedure will be granted to "Object Owner" privilege:

Object Owner	SELECT, INSERT, UPDATE, DELETE, CREATE, DROP, INDEX, ALTER, CREATE TEMPORARY TABLES, LOCK TABLES, EXECUTE, CREATE VIEW, SHOW VIEW, CREATE ROUTINE, ALTER ROUTINE, TRIGGER EXECUTE ON PROCEDURE `gcis_schema`.`flush_db_priv`

Syntax:

```
call GCIS_SCHEMA.CREATE_DB_USER('[User Name]', '[Schema Name]', 'objectowner',
'[Password]');
```

Restrictions:

1. Username can be combination of numeric and alphabetic characters; symbol characters are not allowed
2. Username cannot be the same name to any existing username
3. Username cannot begin with "gcis", "mysql" or "mariadb"
4. Schema name must be an existing DB schema created by B/D
5. Only one user with "create" privilege in each schema. When schema has one user with "create" privilege already, creation of new object owner user will return in error
6. DB user created with object owner initial grant is set to 60 days password expiration
7. Password must be at least 8 characters long
8. Password must contains at least 1 numeric, 1 lower case, 1 upper case and 1 symbol character
9. Length of database username is 80. Refer: https://mariadb.com/kb/en/identifier-names/

Example:

To create an "Object Owner" user with name "app_owner" in the database/schema "testdb1".

```
MariaDB [(none)]>call GCIS_SCHEMA.CREATE_DB_USER('app_owner', 'testdb1', 'objectowner',
'Pass1234%');
+-----------------------------+
| Message                     |
+-----------------------------+
| Success, db user is created. |
+-----------------------------+
1 row in set (0.007 sec)

+-------------------------------------------+
| Message                                   |
+-------------------------------------------+
| Success, privilege 'ObjectOwner' is granted. |
+-------------------------------------------+
1 row in set (0.010 sec)
```

APPENDIX A.5 CREATE DATABASE USER WITH "READ WRITE" PRIVILEGE AND NEVER EXPIRE

The "App Read Write" user is usually for application use. It has 7 schema level privileges:

ReadWrite	SELECT, INSERT, UPDATE, DELETE, CREATE TEMPORARY TABLES, LOCK TABLES, EXECUTE

Syntax:

```
call GCIS_SCHEMA.CREATE_DB_USER('[User Name]', '[Schema Name]', 'app_readwrite',
'[Password]');
```

Restrictions:

1. Username can be combination of numeric and alphabetic characters; symbol characters are not allowed
2. Username cannot be the same name to any existing username
3. Username cannot begin with "gcis", "mysql" or "mariadb"
4. Schema name must be an existing DB schema created by B/D
5. App read-write is initial grant to the user. Object owner user can change the privilege later
6. DB user created with app read-write initial grant is set to never expire
7. Only one never expire user in each schema. When schema has one never expire user already, creation of new app read-write user will return in error
8. Password must be at least 8 characters long
9. Password must contain at least 1 numeric, 1 lower case, 1 upper case and 1 symbol character
10. Length of database username is 80. Refer: https://mariadb.com/kb/en/identifier-names/

Example:

To create an application user with "Read Write" privilege, name "app_user" and never expired account in "testdb1" database/schema.

```
MariaDB [(none)]>call GCIS_SCHEMA.CREATE_DB_USER('app_user', 'testdb1',
'app_readwrite', 'Pass1234%');
+-----------------------------+
| Message                     |
+-----------------------------+
| Success, db user is created. |
+-----------------------------+
1 row in set (0.005 sec)

+-----------------------------------------+
| Message                                 |
+-----------------------------------------+
| Success, privilege 'App_ReadWrite' is granted. |
+-----------------------------------------+
1 row in set (0.007 sec)
```

APPENDIX A.6 CREATE DATABASE USER WITH "READ WRITE" PRIVILEGE

The "Read Write" user is usually for application use. It has 6 schema level privileges:

ReadWrite	SELECT, INSERT, UPDATE, DELETE, LOCK TABLES, EXECUTE

Syntax:

```
call GCIS_SCHEMA.CREATE_DB_USER('[User Name]', '[Schema Name]', 'readwrite',
'[Password]');
```

Restrictions:

1. Username can be combination of numeric and alphabetic characters; symbol characters are not allowed
2. Username cannot be the same name to any existing username
3. Username cannot begin with "gcis", "mysql" or "mariadb"
4. Schema name must be an existing DB schema created by B/D
5. Read-write is initial grant to the user. Object owner user can change the privilege later
6. DB user created with read-write initial grant is set to 60 days password expiration
7. Password must be at least 8 characters long
8. Password must contain at least 1 numeric, 1 lower case, 1 upper case and 1 symbol character
9. Length of database username is 80. Refer: https://mariadb.com/kb/en/identifier-names/

Example:

To create an application user with "Read Write" privilege, name "support_rw".

```
MariaDB [(none)]>call GCIS_SCHEMA.CREATE_DB_USER('support_rw', 'testdb1', 'readwrite',
'Pass1234%');
+-----------------------------+
| Message                     |
+-----------------------------+
| Success, db user is created. |
+-----------------------------+
1 row in set (0.005 sec)

+-----------------------------------------+
| Message                                 |
+-----------------------------------------+
| Success, privilege 'ReadWrite' is granted. |
+-----------------------------------------+
1 row in set (0.007 sec)
```

APPENDIX A.7 CREATE DATABASE USER WITH "READ ONLY" PRIVILEGE

Schema level SELECT privilege will be granted to "Read Only" privilege.

ReadOnly	SELECT

Syntax:

```
call GCIS_SCHEMA.CREATE_DB_USER('[User Name]', '[Schema Name]', 'readonly',
'[Password]');
```

Restrictions:

1. Username can be combination of numeric and alphabetic characters; symbol characters are not allowed
2. Username cannot be the same name to any existing username
3. Username cannot begin with "gcis", "mysql" or "mariadb"
4. Schema name must be an existing DB schema created by B/D
5. Readonly is initial grant to the user. Object owner user can change the privilege later
6. DB user created with read-only initial grant is set to 60 days password expiration
7. Password must be at least 8 characters long
8. Password must contain at least 1 numeric, 1 lower case, 1 upper case and 1 symbol character
9. Length of database username is 80. Refer: https://mariadb.com/kb/en/identifier-names/

Example:

To create a "Read Only" user with name "support_ro" in "testdb1" database/schema.

```
MariaDB [(none)]>call GCIS_SCHEMA.CREATE_DB_USER('support_ro', 'testdb1', 'readonly',
'Pass1234%');
+-----------------------------+
| Message                     |
+-----------------------------+
| Success, db user is created. |
+-----------------------------+
1 row in set (0.007 sec)

+-----------------------------------------+
| Message                                 |
+-----------------------------------------+
| Success, privilege 'ReadOnly' is granted. |
+-----------------------------------------+
1 row in set (0.010 sec)
```

APPENDIX A.8 CREATE DATABASE USER WITH "CONNECT ROLE" ONLY

In case of need, B/D user can create a database user with "Connect Role" only. The "Object Owner" user can further grant other object level privileges to this user.

Syntax:

```
call GCIS_SCHEMA.CREATE_DB_USER('[User Name]', '', 'connect', '[Password]');
```

Restrictions:

1. Username can be combination of numeric and alphabetic characters; symbol characters are not allowed
2. Username cannot be the same name to any existing username
3. Username cannot begin with "gcis", "mysql" or "mariadb"
4. DB user created with connect initial grant is set to 60 days password expiration
5. Password must be at least 8 characters long
6. Password must contain at least 1 numeric, 1 lower case, 1 upper case and 1 symbol characters
7. Length of database username is 80. Refer: https://mariadb.com/kb/en/identifier-names/

Example:

To create user with "Connect Role" only. Please note that the schema field is an empty string.

```
MariaDB [(none)]>call GCIS_SCHEMA.CREATE_DB_USER('support_user', '', 'Connect',
'Pass1234%');
+-----------------------------+
| Message                     |
+-----------------------------+
| Success, db user is created. |
+-----------------------------+
1 row in set (0.005 sec)

+----------------------------------------+
| Message                                |
+----------------------------------------+
| Success, privilege 'Connect' is granted. |
+----------------------------------------+
1 row in set (0.006 sec)
```

APPENDIX A.9 LIST DATABASE USERS

List all users created by B/D.

Syntax:

```
call GCIS_SCHEMA.LIST_DB_USER();
```

Example:

```
MariaDB [(none)]>call GCIS_SCHEMA.LIST_DB_USER();
+--------------+--------------+-------------------------+
| User         | never expire | password_expiration_time |
+--------------+--------------+-------------------------+
| app_user     | Yes          |                         |
| app_owner    | No           | 2021-09-09 22:50:36     |
| support_ro   | No           | 2021-09-09 22:50:55     |
| support_rw   | No           | 2021-09-09 22:53:21     |
| support_user | No           | 2021-09-09 22:53:27     |
+--------------+--------------+-------------------------+
5 rows in set (0.003 sec)
```

APPENDIX A.10 CHANGE DATABASE USER PASSWORD BY DB ADMIN USER

Syntax:

```
call GCIS_SCHEMA.CHNAGE_DB_USER_PASSWORD('[User Name]', '[New Password]');
```

Restrictions:

1. Username must be an existing DB user created by B/D
2. Password must be at least 8 characters long
3. Password must contain at least 1 numeric, 1 lower case, 1 upper case and 1 symbol character

Example:

```
MariaDB [(none)]>call GCIS_SCHEMA.CHANGE_DB_USER_PASSWORD('app_owner', 'New02468%');
+------------------------------+
| Message                      |
+------------------------------+
| Success, new password is set. |
+------------------------------+
1 row in set (0.007 sec)
```

APPENDIX A.11 CHANGE DATABASE USER PASSWORD BY USER ITSELF

Syntax:

```
SET PASSWORD = PASSWORD('[New Password]') ;
```

Restrictions:

1. Password must be at least 8 characters long
2. Password must contain at least 1 numeric, 1 lower case, 1 upper case and 1 symbol character

Example:

```
MariaDB [(none)]>set PASSWORD = PASSWORD('New02468%');
Query OK, 0 rows affected (0.001 sec)
```

APPENDIX A.12 DROP DATABASE USER

When dropping “Object Owner” database a/c, database objects created by specified database a/c will NOT be cascade deleted

Syntax:

```
call GCIS_SCHEMA.DROP_DB_USER('[User Name]');
```

Restrictions:

1. Must be existing DB user
2. Cannot drop DB system user

Example:

```
MariaDB [(none)]>call GCIS_SCHEMA.DROP_DB_USER('app_owner');
+---------------------------+
| Message                   |
+---------------------------+
| Success, user is dropped. |
+---------------------------+
1 row in set (0.007 sec)
```

APPENDIX A.13 LIST DATABASE USER SESSIONS

Syntax:

```
call GCIS_SCHEMA.LIST_DB_SESSION();
```

Example:

```
MariaDB [(none)]>call GCIS_SCHEMA.LIST_DB_SESSION();
+-------+-----------+---------+--------------------+---------+-------+------+
| id    | user      | db      | host               | command | state | info |
+-------+-----------+---------+--------------------+---------+-------+------+
| 28632 | app_owner | testdb1 | ttua-dbpxy04:37670 | Sleep   |       | NULL |
| 28633 | support_rw | testdb1 | ttua-dbpxy04:37672 | Sleep   |       | NULL |
+-------+-----------+---------+--------------------+---------+-------+------+
2 rows in set (0.004 sec)
```

APPENDIX A.14 KILL DATABASE USER SESSIONS

Syntax:

```
call GCIS_SCHEMA.KILL_DB_SESSION(['Session ID]');
```

Restrictions:

1. Cannot kill DB system session

Example:

```
MariaDB [(none)]>call GCIS_SCHEMA.KILL_DB_SESSION(28632);
+----------------------------+
| Message                    |
+----------------------------+
| Success, session is killed. |
+----------------------------+
1 row in set (0.014 sec)
```

APPENDIX A.15 LIST DATABASE LOCKING

Syntax:

```
call GCIS_SCHEMA.SHOW_DB_LOCK();
```

Example:

```
MariaDB [(none)]>call GCIS_SCHEMA.SHOW_DB_LOCK();
+---------+---------------------+---------+------------+------------+-----------------
---+-----------+
| LOCK_ID | LOCK_STARTED_TIME   | SCHEMA  | TABLE_NAME | LOCK_STATE | LOCK_TYPE
| SQL_QUERY |
+---------+---------------------+---------+------------+------------+-----------------
---+-----------+
|   77912 | 2021-07-20 12:25:20 | testdb1 | t1         | RUNNING    | Table metadata lock
| NULL      |
+---------+---------------------+---------+------------+------------+-----------------
---+-----------+
1 row in set (0.001 sec)

Query OK, 0 rows affected (0.001 sec)
```

APPENDIX A.16 LIST DATABASE BLOCKING

Syntax:

```
call GCIS_SCHEMA.SHOW_DB_BLOCK();
```

Example:

```
MariaDB [(none)]>call GCIS_SCHEMA.SHOW_DB_BLOCK();
+--------------------+--------------------+------------+--------------------------
------+
| BLOCKING_SESSION_ID | LOCK_TYPE          | LOCK_OBJECT | LOCK_SQL
|
+--------------------+--------------------+------------+--------------------------
------+
|              28572 | Table metadata lock | t1         | update t1 set c1=-1 where
c1=100 |
+--------------------+--------------------+------------+--------------------------
------+
1 row in set (0.001 sec)

Query OK, 0 rows affected (0.001 sec)
```

APPENDIX A.17 FLUSH DATABASE PRIVILEGE

Syntax:

```
call GCIS_SCHEMA.FLUSH_DB_PRIV();
```

Example:

```
MariaDB [gcis_schema]> call GCIS_SCHEMA.FLUSH_DB_PRIV();
Query OK, 0 rows affected (0.005 sec)
```

APPENDIX A.18 FLUSH HOST CACHE

Syntax:

```
call GCIS_SCHEMA.FLUSH_DB_HOST();
```

Example:

```
MariaDB [gcis_schema]> call GCIS_SCHEMA.FLUSH_DB_HOST();
Query OK, 0 rows affected (0.006 sec)
```